

prog2 lab 2025

Correttezza di un programma e TDD

Riassunto (mie opinioni)

- Quando scrivete un programma, esso deve fare quello che e' scritto nei requisiti

Per verificarlo si usa il **testing**

- Deve essere inoltre comprensibile/modificabile da un essere umano per la manutenzione

Per realizzare questo si usa la **documentazione**

Documentazione, argomento vastissimo

- Al di là della qualità della documentazione, il problema principale è la sua manutenzione per tutta la durata del progetto
- Spesso codice e doc tendono a divergere, rendendola fuorviante, meglio non esagerare quindi
- Sul mondo del lavoro ogni ditta può avere standard e strumenti diversi
- Lo stesso committente del progetto può richiedere l'utilizzo di determinati standard

Correttezza e metodi formali

- Come forse avrete visto, si puo' immaginare che un programma sia una funzione di cui si possa verificare la correttezza
- Purtroppo la verifica formale e' troppo difficile da applicare a funzioni che non siano banali
- A quel punto tanto vale impegnarsi e sviluppare con maggior cura il codice

Predicati input ed output e invariante di ciclo

- Al di là del formalismo I concetti sono utili nella documentazione:
- Per descrivere il comportamento di una funzione
- Per chiarire allo stesso programmatore quali siano le parti importanti del funzionamento di un ciclo.

for e while

- se riflettiamo un attimo, il for sembra piu' "ordinato" del while
- for (inizializzazioni varie, condizione di ciclo, iterazione) {}
- ci evidenzia bene quali sono le variabili che lo riguardano e la "condizione di ciclo" puo' avere delle relazioni con l'"invariante di ciclo", o almeno aiutarci a scriverla se serve.
- nel while... invece i vari componenti sono sparsi nel codice e ne possono rendere piu' difficile la comprensione
- ovviamente in certi casi e' vero il contrario...

for e while

```
int val = 0;
for(char* curr = start; *curr != '\0'; curr++, val++) {
    if(*curr == c) {
        return val;
    }
}
```

```
int val = 0;
char* curr = start;
while( *curr != '\0') {
    if(*curr++ == c) { ← molto usato in C ma modifico var del ciclo!
        return val;
    };
    val++;
}
```

Usare I test????

- Se ci pensiamo bene, come si possono vedere I test?
- Come degli esempi di comportamento del programma e quindi aiutano a capire che cosa “fa” una funzione.
- Questo e' utile perche' mentre per una funzione ad alto livello “metti item nel carrello cliente” il significato e' evidente, spesso le funzioni a livello piu' basso possono avere parti meno comprensibili.
- Inoltre I test devono **obbligatoriamente** essere aggiornati e coerenti col programma testato
- Scrivere una funzione pensando ai test puo' anche stimolare a chiarirsi bene le idee e scrivere codice migliore

Che faticaccia

- Scrivere test per tutte le funzioni e' oneroso, noioso...
- E' un investimento per il futuro del progetto
- Piu' e' grande e lungo e piu' diventa importante
- Ovviamente non e' obbligatorio fare gli automi, a volte se si prototipa una idea si ha fretta di capire se funzionerebbe ..

Idea generale di Compromesso

- Ogni volta che scrivete una funzione, iniziate a predisporre nello stesso modulo (o uno gemello) una compagna di testing:

```
Int funzione(int p1,int p2..){}
```

```
Int test_funzione(int p1,int p2...){
```

```
////  setup
```

```
int val = funzione(p1,p2,..);
```

```
//Verifica val e return risultato test
```

```
}
```

Idea generale di Compromesso

- Fatto questo, create una funzione generale di test che richiama le test_*(())
- Questa funzione, se possibile, dovrà essere richiamata ogni volta che modificate il codice e ricompilate
- Cosa manca? Dovete preparare gli argomenti di input/output alla funzione ed almeno un test. Qui dipende dalla funzione e vedremo piu' avanti
- NB questa e' una strategia generale, la sua implementazione variera' a seconda del "committente" che potrebbe richiedervi uno standard particolare di nomi, organizzazione dei sorgenti etc etc, ma i concetti sono sempre questi
- quando si utilizza **CodeRunner**, dietro alle quinte lavora una struttura simile a questa

Idea generale di Compromesso

- Una volta che avete preparato tutto il setup, l'aggiunta di una nuova funzione con test relativo e' rapida e siete meno invogliati a rimandarla
- Un altro vantaggio e' che aggiungere un nuovo test e' rapidissimo, mentre se la funzione e' annidata nel programma puo' essere complicato, anche se si volesse, creare nel programma le condizioni per farla agire come nel test che vorremmo.
- Come procedere? Come intuibile conviene procedere Bottom-Up, in modo che le funzioni a basso livello siano "affidabili" e richiamabili con tranquillita' da quelle sopra.
- se una funzione a basso livello e' errata, e' costoso individuarla
- Questo non risolve tutto, vedrete in altri corsi...

Test Driven Development

- come avrete visto a teoria, per scrivere un programma occorre avere una idea chiara di quello che dovete realizzare
- facile a dirsi!!!
- alcune persone sono nate “programmatore”, noi comuni mortali faticiamo.
- per facilitarci il compito si parte da un abbozzo dell’idea che si sviluppa per raffinamenti (cicli) e prototipi successivi.
- E’ importante , man mano che si procede, assicurarsi che la nostra visione del problema e tutto il lavoro fatto finora siano corretti; prima si rileva un errore e meno costa correggerlo, con progressione molto piu’ che lineare..

Test Driven Development

- Per realizzare questo si può ricorrere alla metodologia detta appunto Test Driven Development:
- Partiamo da un esempio semplice: voglio creare e testare una funzione che calcoli e restituisca il quadrato di un numero:

```
int quadrato(int x);
```

scrivo delle coppie di valori input output:

0 → 0

1 → 1

10 → 100

-1 → 1

Test Driven Development

Preparo poi un programma che esegua la funzione per ogni coppia di dati e stampi se coincidono o meno

fail =0

for x,y in coppievaloritest:

 print("testing", x, y)

 risultato = quadrato(x) // richiamo

 if risultato != y:

 print("fail",risultato)

 fail +=1

 else:

 print("pass")

return fail

inciso: Funzione testabile?

- Ovviamente, a seconda dei parametri I/O , confrontare puo' essere piu' o meno semplice
- se avete degli interi e' banale, se avete tipi complessi e' piu' laborioso, magari dovere scrivere delle funzioni..
- In molti casi e' comodo avere, per ogni struttura dati, due funzioni

```
char * to_string(miotipo *var){...}  
miotipo * from_string(char *s){...}
```

che trasformi da/a stringa il tipo

Funzione testabile?

- questo permette di dare una rappresentazione uniforme ai risultati dei test, quindi la test suite non dovrà essere modificata nella struttura se non per casi particolari
- notate che dotare i tipi di queste funzioni di supporto ci porta verso il paradigma della programmazione ad oggetti, dove ogni dato è corredato dalle funzioni che operano su di esso.
- anche se in C non esistono gli Oggetti, i concetti utili si possono simulare caso per caso

Nuovo ciclo (pseudocodice)

assumo che X e Y siano stringhe (char*)

for x,y in coppievaloritest:

 print("testing", x, y)

 risultato = quadrato(**from_string**(x))

 if **to_string**(risultato) != y:

 print("fail",risultato)

 else:

 print("pass")

Torniamo all'esempio

Supponiamo adesso che il programma passi tutta la test suite, ma qualcuno rilevi un errore per questo caso:

se $x = 2^{**}30$ allora y dovrebbe valere $2^{**}60$

questo spesso e' un OVERFLOW a seconda dell' HW

- Allora, ogni volta che qualcuno trova un errore, occorre aggiungere ai test il caso relativo, che si chiama test di **regressione**
- In programmi complessi e molto in la' nella manutenzione, spesso capita che una modifica riesumi errori passati, quindi i test di regressione sono fondamentali
- Un compilatore C puo' avere una test suite di piu' di 20.000 casi!

Test Driven Development

- Questa e' l'idea di base, che puo' essere implementata artigianalmente o con tool appositi che supportano il testing, integrati con gli ambienti di sviluppo (IDE)
- Il ciclo di sviluppo sintetico diventa quindi:
 - a. scrivi un requisito
 - b. scrivi un test
 - c. scrivi codice per passare il test
 - d. fai girare il test e tutti quelli precedenti
 - e. riorganizza il codice!!**
 - f. rifai girare il test e tutti quelli precedenti

Si raccomanda di far girare i test ad ogni modifica, per quanto innocua possa apparire

riorganizzare il codice

- Ogni volta che il vostro programma raggiunge un qualche obiettivo, e' fondamentale riorganizzare il codice.
 - fattorizzare in funzioni codice duplicato
 - generalizzare le funzioni
 - raggruppare dati e funzioni in moduli "coerenti"
-

Come sempre, i vantaggi aumentano rapidamente all'aumentare delle dimensioni del progetto.

Test Driven Development

Perche' ci guadagniamo?

Si evita di perdere tempo in debugging, (se testo in continuazione) perche' ogni piccola modifica e' testata subito, mentre se testo raramente ci sono tante interazioni possibili tra le modifiche

Quando funziona meglio?

ovviamente quando l' output di un programma e' facile da confrontare con quello "corretto", pensate invece ad esempio ad una GUI o un programma che dipende da eventi temporali.

Test Driven Development

Utile per le funzioni a basso livello:

Sono le basi del sistema e si spera che una volta implementate non dovranno piu' essere modificate ma devono garantire funzionamento corretto ai moduli superiori.

Hanno spesso un unico output corretto per ogni input

Hanno un numero di casi limitato

Richiede un investimento iniziale di setup, che si ripaga piu' avanti nel progetto